

# ZizkaDB — Developer Technical Brief

---

# ZizkaDB — Developer Technical Brief

---

**Audience:** Marketing / sales talking to software engineers, ML engineers, and agent builders.

**Purpose:** Answer technical questions accurately without overstating the product.

**Last updated:** May 2026

**Product URL:** <https://db.zizka.ai>

**Docs:** <https://db.zizka.ai/docs>

**API explorer:** <https://db.zizka.ai/api-explorer>

**GitHub:** <https://github.com/Zizka-ai/agentdb>

---

## 1. One-sentence pitch (memorize this)

**ZizkaDB is the operational database for AI agents** — it logs what agents do, links decisions by cause, searches history by meaning, replays past state, and tells you when an agent’s behavior has drifted from its own baseline.

---

## 2. What ZizkaDB is — and is not

| ZizkaDB is                                                                   | ZizkaDB is not                                          |
|------------------------------------------------------------------------------|---------------------------------------------------------|
| An <b>event store + memory layer</b> for agents                              | A vector DB you query only by embedding similarity      |
| <b>Framework-agnostic</b> (Claude, OpenAI, LangChain, custom code)           | Locked to one LLM vendor or one orchestration framework |
| <b>Causal</b> (parent → child event links)                                   | A chat transcript dump                                  |
| <b>Self-hostable</b> (Docker, full features, free)                           | SaaS-only                                               |
| <b>Model-agnostic</b> for logging; embeddings use OpenAI for semantic search | A replacement for your LLM                              |

**Analogy for developers:** “Think Postgres for agent *decisions*, plus lineage, semantic recall, and drift detection — not another Pinecone index.”

---

### 3. The problem we solve

When you ship agents to production, teams hit the same walls:

1. **"Why did it do that?"** — Tool calls and replies are disconnected; you cannot walk backward from a bad output to the input that caused it.
2. **"What did it know at the time?"** — Chat logs are not reconstructible agent *state* at a timestamp.
3. **"Is this agent broken or just different?"** — Prompt changes, model upgrades, and tool additions change behavior silently.
4. **"Our memory is a black box"** — Vendor memory (Claude/OpenAI) does not live in *your* codebase, audit trail, or GDPR erasure story.

ZizkaDB is built for **production debugging, compliance, and cross-session memory you control**.

---

### 4. Core capabilities (what to say to engineers)

#### 4.1 Event logging

Everything is an **event**: `agent` name, `event` type string, JSON `data` payload, timestamp.

```
result = await db.log(  
    agent="support-bot",  
    event="tool_call",  
    data={"tool": "get_billing", "user_id": 123},  
)  
# result.event_id – use for causal links and why()
```

Optional: `session_id` (groups a conversation/run), `metadata` (tenant-defined tags).

#### 4.2 Causal lineage

Pass `parent_id` (Python) / `parentId` (TypeScript) to say *this event happened because of that one*.

```
msg = await db.log(agent="bot", event="user_message", data={...})  
tool = await db.log(agent="bot", event="tool_call", data={...}, parent_id=msg.event_id)  
chain = await db.why(tool.event_id)  
chain.print() # walks the tree to the root
```

**API:** GET `/v1/events/{event_id}/why`

#### 4.3 Semantic search

Events are embedded automatically (**OpenAI** `text-embedding-3-small`). Search in natural language across an agent's history.

```
results = await db.search("customer angry about billing", agent="support-bot", limit=10)
```

**API:** POST /v1/search

## 4.4 Time travel

Reconstruct agent state at any past timestamp from logged events. Each event has a **SHA-256 checksum** over payload content.

```
state = await db.at("support-bot", datetime(2026, 5, 1, 15, 0))
```

**API:** GET /v1/events/at

## 4.5 Context injection (memory for prompts)

Drop-in style memory for system prompts: recent events + semantically relevant past events, trimmed to a token budget.

```
context = await db.context_for(agent="support-bot", task="user asking about billing",  
                               max_tokens=2000)
```

**API:** POST /v1/memory/context

## 4.6 Behavioral baseline & drift

After enough sessions (with `session_id` on logs), ZizkaDB compares **recent** behavior vs **historical baseline**:

- Event-type distribution
- Parent→child transition patterns
- Session length, error rate

Returns a **drift score** (0 = identical, 1 = totally different) and verdict: `stable`, `minor_drift`, `noticeable_drift`, `significant_drift`.

```
baseline = await db.baseline(agent="support-bot", recent_window=50)
```

**API:** GET /v1/agents/{agent\_id}/baseline

**Honest caveat:** Needs real session volume; early on status is `warming_up` or `insufficient_data`. This is statistical behavior comparison, not “the model hallucinated” detection.

## 4.7 Session diff & GDPR forget

- `memory_diff(session_id)` — what changed in a session (API: GET /v1/memory/diff/{session\_id}).

- `forget(filter_key, filter_value)` — delete all events matching metadata (e.g. `user_id` , `email` ) for GDPR erasure (API: `DELETE /v1/memory/forget` ).

### 4.8 Audit trail

Events are **append-only** with per-event checksums. Position as tamper-evident operational history (export story is on roadmap/marketing — verify with engineering before promising signed legal exports).

## 5. How developers integrate (four paths — same backend)

| Path           | Install                              | Best for                                                      |
|----------------|--------------------------------------|---------------------------------------------------------------|
| Python SDK     | <code>pip install zizkadb-sdk</code> | FastAPI agents, notebooks, batch jobs                         |
| TypeScript SDK | <code>npm install zizkadb-sdk</code> | Node, Bun, Deno, edge workers                                 |
| MCP server     | <code>uvx zizkadb-mcp</code>         | Claude Desktop, Cursor, Windsurf — <b>no app code changes</b> |
| REST API       | <code>curl</code> / any HTTP client  | Go, Rust, Java, Ruby, mobile                                  |

#### Cloud (managed):

```
from zizkadb import ZizkaDB
db = ZizkaDB("agdb_live_xxxx") # API key from dashboard Settings
```

#### Self-hosted:

```
db = ZizkaDB(host="http://localhost:8000") # or DEV_API_KEY for local dev
```

Default cloud host in SDKs: `https://db.zizka.ai`

## 6. MCP — the “zero code” pitch for Cursor/Claude users

**MCP** = Model Context Protocol. ZizkaDB ships an MCP server so the IDE/assistant can call tools directly.

**Config example (Cursor `~/.cursor/mcp.json` or Claude Desktop):**

```
{
  "mcpServers": {
    "zizkadb": {
      "command": "uvx",
      "args": ["zizkadb-mcp"],
      "env": { "ZIZKADB_API_KEY": "agdb_live_xxxx" }
    }
  }
}
```

**Self-hosted MCP:** set `ZIZKADB_HOST` to your API URL.

| MCP tool                   | What it does                           |
|----------------------------|----------------------------------------|
| <code>log_event</code>     | Log action with optional causal parent |
| <code>search_memory</code> | Semantic search over history           |
| <code>get_context</code>   | Prompt-ready memory block              |
| <code>why</code>           | Causal chain for an event              |
| <code>query_events</code>  | List/filter recent events              |
| <code>time_travel</code>   | State at a timestamp                   |
| <code>memory_diff</code>   | Session summary                        |
| <code>forget</code>        | GDPR delete by filter                  |

**MCP license:** MIT (SDK/core stack: AGPL-3.0 — see §12).

## 7. REST API map (for “show me the HTTP” questions)

Base URL: `https://db.zizka.ai` (self-host: your host, port **8000**).

**Auth:** `Authorization: Bearer <token>`

- Production API keys: prefix `agdb_live_` (or `agdb_` generally)
- Dashboard login uses JWT (OTP email)
- Self-host dev: optional `DEV_API_KEY` in `.env` (never in production)

| Area   | Method | Path                             | Purpose      |
|--------|--------|----------------------------------|--------------|
| Events | POST   | <code>/v1/events</code>          | Log event    |
| Events | GET    | <code>/v1/events</code>          | Query events |
| Events | GET    | <code>/v1/events/{id}/why</code> | Causal chain |

| Area   | Method | Path                         | Purpose                            |
|--------|--------|------------------------------|------------------------------------|
| Events | GET    | /v1/events/at                | Time travel                        |
| Search | POST   | /v1/search                   | Semantic search                    |
| Memory | POST   | /v1/memory/context           | Context block                      |
| Memory | GET    | /v1/memory/diff/{session_id} | Session diff                       |
| Memory | DELETE | /v1/memory/forget            | GDPR erase                         |
| Agents | GET    | /v1/agents                   | List agents                        |
| Agents | GET    | /v1/agents/{id}/baseline     | Drift / baseline                   |
| Agents | GET    | /v1/agents/{id}/sessions     | Sessions                           |
| Auth   | POST   | /v1/auth/request-otp         | Email OTP                          |
| Auth   | POST   | /v1/auth/verify-otp          | Get JWT                            |
| Auth   | POST   | /v1/auth/api-keys            | Create API key                     |
| Health | GET    | /health                      | Status check                       |
| Stats  | GET    | /v1/stats                    | Public install counters (homepage) |

Interactive docs: <https://db.zizka.ai/api-explorer> (FastAPI OpenAPI).

## 8. SDK method cheat sheet

| Method                                | Purpose         |
|---------------------------------------|-----------------|
| log(...)                              | Write event     |
| why(event_id)                         | Causal chain    |
| search(query, ...)                    | Semantic search |
| at(agent, timestamp)                  | Time travel     |
| query(agent, ...)                     | List events     |
| context_for(agent, task, max_tokens?) | Prompt memory   |
| baseline(agent, recent_window?)       | Drift detection |
| memory_diff(session_id)               | Session summary |
| forget(filter_key, filter_value)      | GDPR delete     |
| agents()                              | List agents     |

Python 3.10+, TypeScript works on Node/Deno/Bun/edge.

---

## 9. Architecture (for technical due diligence)

### Managed production (db.zizka.ai):

- **Dashboard:** Next.js 14 (PM2 on port 3001 behind nginx)
- **API:** FastAPI (Python), Docker, port 8000
- **Postgres + pgvector** — events, tenants, embeddings
- **Qdrant** — vector search
- **Redis** — caching/sessions
- **Nginx** — TLS, routes `/` → frontend, `/v1/` → API

### Self-host (Docker Compose):

```
git clone https://github.com/Zizka-ai/agentdb
cp .env.example infra/.env # OPENAI_API_KEY required for embeddings
docker compose -f infra/docker-compose.yml up -d
```

Stack: postgres (pgvector), qdrant, redis, api. Dashboard optional ( `npm run dev` in `dashboard/` ).

**Embeddings:** Without `OPENAI_API_KEY`, **logging still works**; semantic search and context injection that rely on embeddings will not.

**Telemetry:** SDK/MCP send **one anonymous ping** on startup (SDK name, version, OS, cloud vs self-host). No event payloads, no API keys. Opt out: `AGENTDB_TELEMETRY=false` or `ZIZKADB_TELEMETRY=false` (confirm env name in release notes).

---

## 10. Authentication & onboarding flow

1. Sign up at <https://db.zizka.ai/signup> — email OTP, no password.
2. Dashboard → **Settings** → **Create API key** ( `agdb_live_...` ).
3. Paste key into SDK, MCP env, or `Authorization: Bearer` header.
4. Start logging from agent code or MCP.

**Multi-tenant:** API keys are scoped to a tenant/project; events are isolated per tenant.

---

## 11. Pricing (managed cloud)

| Plan        | Price        | Highlights                                                                                                    |
|-------------|--------------|---------------------------------------------------------------------------------------------------------------|
| Self-Hosted | Free forever | Full feature set, your infra, Docker, community support                                                       |
| Pro         | €29/mo       | 100M events, 90-day retention, 3 projects, email support, <b>1-month free trial</b> , no credit card to start |
| Team        | €99/mo       | 1B events, 1-year retention, 10 seats, priority support, 1-month free trial                                   |

**Line for developers:** “Free if you self-host; paid if you don’t want to run Postgres/Qdrant yourself.”

## 12. Licensing (do not hand-wave this)

| Component                  | License                                |
|----------------------------|----------------------------------------|
| Core API + self-host stack | <b>AGPL-3.0</b>                        |
| Python SDK                 | <b>AGPL-3.0</b>                        |
| TypeScript SDK             | <b>AGPL-3.0</b> (per package metadata) |
| MCP server                 | <b>MIT</b>                             |

**What to say:** “Open source, self-hostable. AGPL applies to the core platform and SDKs — if you modify and distribute the server, AGPL obligations apply. MCP is MIT for easy IDE integration. Legal review for commercial embedding is normal.”

**Do not say:** “Fully MIT” or “no license restrictions.”

## 13. Competitive positioning (facts-first)

Use the comparison on the homepage; verify competitors’ docs before live debates.

| Capability                 | LangSmith | Mem0 | Pinecone | ZizkaDB |
|----------------------------|-----------|------|----------|---------|
| Agent event logging        | ✓         | ×    | ×        | ✓       |
| Causal lineage             | ~         | ×    | ×        | ✓       |
| Time travel (state at T)   | ×         | ×    | ×        | ✓       |
| Semantic search on history | ×         | ✓    | ✓        | ✓       |
| Any framework/model        | ~         | ✓    | ✓        | ✓       |

| Capability                  | LangSmith | Mem0 | Pinecone | ZizkaDB |
|-----------------------------|-----------|------|----------|---------|
| Behavioral baseline / drift | ×         | ×    | ×        | ✓       |
| Cross-agent fleet queries   | ×         | ×    | ×        | ✓       |
| Tamper-evident audit        | ×         | ×    | ×        | ✓       |
| Self-host free              | ✓         | ✓    | ×        | ✓       |

~ = partial (e.g. LangSmith is strongest inside LangChain ecosystem).

**ZizkaDB wedge:** Vector DBs store embeddings; ZizkaDB stores **what happened, why, and what came next**, plus **drift vs the agent's own past**.

---

## 14. FAQ — hard questions from developers

### Q: How is this different from LangSmith?

A: LangSmith is excellent for LangChain-centric tracing and evals. ZizkaDB is a **standalone operational DB** with causal trees, time travel, cross-agent queries, and behavioral baselines — works with any stack.

### Q: How is this different from Mem0?

A: Mem0 focuses on long-term memory retrieval for prompts. ZizkaDB adds **causal lineage, session replay, drift baselines, and audit-oriented event storage**.

### Q: Do we need to replace Pinecone?

A: No. Pinecone is for RAG document search. ZizkaDB is for **agent decision history**. Many teams use both.

### Q: Does it wrap our LLM calls?

A: No. You add `db.log()` (or MCP `log_event`) where you already handle messages/tools. One line for causal link: `parent_id=...`.

### Q: Latency impact?

A: Logging is async HTTP; embedding generation is non-blocking on the hot path. Typical integration is fire-and-forget `await db.log(...)`.

### Q: What if OpenAI is down / we don't use OpenAI?

A: Logging and causal features work without embeddings. Semantic search and `context_for` need embeddings (today: OpenAI). Other embedding providers = roadmap; say "check with team" unless confirmed shipped.

### Q: PII / GDPR?

A: You control what goes in `data` and `metadata`. `forget()` deletes by metadata filter. Self-host keeps data on your infra.

**Q: Can we use it in regulated environments?**

A: Self-host + your VPC is the usual answer. Managed cloud: discuss DPA, retention, and region with founders — don't invent certifications.

**Q: PyPI package confusion?**

A: Install `zizkadb-sdk` (not the unrelated `agentdb` package on PyPI). Import: `from zizkadb import ZizkaDB`.

---

## 15. Integration snippets (copy for demos)

**Minimal Python (managed):**

```
import asyncio
from zizkadb import ZizkaDB

async def main():
    db = ZizkaDB("agdb_live_xxxx")
    r = await db.log(agent="demo-bot", event="started", data={"env": "prod"})
    print(await db.why(r.event_id))

asyncio.run(main())
```

**Claude / OpenAI agents:** Log `user_message`, each `tool_call`, and `assistant_response` with `parent_id` chaining — see homepage examples at [db.zizka.ai](https://db.zizka.ai).

**curl:**

```
curl -X POST https://db.zizka.ai/v1/events \
-H "Authorization: Bearer agdb_live_xxxx" \
-H "Content-Type: application/json" \
-d '{"agent":"demo","event":"tool_call","data":{"tool":"search"}}'
```

---

## 16. What marketing should not claim (yet)

- “Real-time alerts” everywhere — baseline exists; **alerting product** may be partial/roadmap (homepage mentions “Alerts coming” for session 50 narrative).
- “Replaces your observability stack” — it complements APM/logs; different layer.
- “Detects hallucinations” — drift is **behavioral statistics**, not truth verification.
- “SOC2 / HIPAA certified” — unless explicitly launched.
- “Works offline” — needs API reachability (or self-host on localhost).

When unsure: **“Let me connect you to engineering”** or point to **docs + GitHub**.

---

## 17. Elevator pitches by persona

| Persona                  | Pitch                                                                                                             |
|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| Backend engineer         | "One <code>log()</code> per decision, <code>parent_id</code> for causality, <code>why()</code> when prod breaks." |
| AI/ML engineer           | "Semantic search + time travel + drift baselines without locking to LangChain."                                   |
| Platform / infra         | "Self-host Postgres+Qdrant stack, AGPL, Docker in 60 seconds."                                                    |
| Cursor/Claude power user | "Add MCP — your IDE gets memory, search, and <code>why()</code> with no SDK refactor."                            |
| CTO / head of AI         | "Operational DB for agents: audit trail, GDPR forget, fleet baselines — not another vector index."                |

## 18. Links & contacts

| Resource     | URL                                                                                   |
|--------------|---------------------------------------------------------------------------------------|
| Product      | <a href="https://db.zizka.ai">https://db.zizka.ai</a>                                 |
| Sign up      | <a href="https://db.zizka.ai/signup">https://db.zizka.ai/signup</a>                   |
| Docs         | <a href="https://db.zizka.ai/docs">https://db.zizka.ai/docs</a>                       |
| API explorer | <a href="https://db.zizka.ai/api-explorer">https://db.zizka.ai/api-explorer</a>       |
| GitHub       | <a href="https://github.com/Zizka-ai/agentdb">https://github.com/Zizka-ai/agentdb</a> |
| Company      | Zizka AI                                                                              |

## 19. Quick reference card (print this)

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| Product:    | ZizkaDB — operational DB for AI agents                                           |
| URL:        | <code>db.zizka.ai</code>                                                         |
| Install:    | <code>pip install zizkadb-sdk   npm install zizkadb-sdk   uvx zizkadb-mcp</code> |
| Key prefix: | <code>agdb_live_</code>                                                          |
| Core APIs:  | <code>log · why · search · at · context_for · baseline · forget</code>           |
| Different:  | <code>causal lineage · time travel · behavioral drift · self-host free</code>    |
| Not:        | <code>vector-only DB · LangChain-only · LLM wrapper</code>                       |
| License:    | <code>AGPL (core/SDK) · MIT (MCP)</code>                                         |

*Internal doc for Zizka AI marketing. For engineering changes, verify against `main` branch and live docs before external publication.*